Derivative-Free Optimization for Black-Box Functions in Hyperparameter Tuning

Fiorella Yannet Paredes Coaguila*, Teresa Paola Álvarez Rozas†
*Universidad Nacional del Altiplano - FINESI
Email: 75959821@est.unap.edu.pe
†Universidad Nacional del Altiplano - FINESI
Puno, Peru

***Abstract*—Hyperparameter optimization in machine learning algorithms represents a significant computational challenge due to the black-box nature of objective functions. Machine learning algorithms are complex black boxes that create challenging optimization problems, with non-smooth, discontinuous objective functions and unpredictable variations in computational cost. This study evaluates four derivative-free optimization methods: Tree-structured Parzen Estimator (TPE), Random Search, Covariance Matrix Adaptation Evolution Strategy (CMA-ES), and Quasi-Monte Carlo (QMC) for hyperparameter tuning in Random Forest and XGBoost models. Using the House Prices dataset, controlled experiments were conducted with 50 evaluations per method and 3 independent runs. The results demonstrate that TPE, as a Bayesian optimization algorithm, enables principled updating of initial beliefs about optimal hyperparameters and achieved the best average performance with an RMSE of 29,803.67 ± 316.86 for Random Forest. Statistical analysis revealed significant differences between methods, validating TPE's superiority over traditional methods like Random Search for this type of optimization problems.**

***Index Terms*—derivative-free optimization, hyperparameter tuning, black-box functions, TPE, machine learning**

I. Introduction

Hyperparameter tuning is a fundamental part of creating machine learning models. Their correct selection depends not only on how fast a model is trained, but also on how good its final results are. In general, hyperparameters directly influence training efficiency and the quality of the resulting model. [1].

The problem is complicated because many times these evaluation functions behave like true "black boxes" - we don't know exactly what happens inside them or how they react to changes. Additionally, hyperparameters can be very varied - from continuous numbers to categories or integer values - and it's not uncommon for some tests to fail due to technical errors or unexpected problems [2].

Traditionally, the most common strategy for optimizing hyperparameters has been grid search, which consists of testing all possible combinations within a predefined set. However, this approach presents significant limitations in terms of computational efficiency and scalability [12].

Faced with these limitations, more modern methods have emerged, such as derivative-free optimization, which have the advantage of not requiring information about how the function being evaluated changes. Among the most prominent methods is the Tree-structured Parzen Estimator (TPE), which models which combinations have worked well or poorly, and focuses on testing where there is a higher probability of improvement. This approach has been the subject of recent research seeking to understand how each of its components influences its good performance. Another relevant method is CMA-ES, an evolutionary strategy inspired by natural selection, which adjusts its solutions progressively and adaptively [4].

Likewise, TPE has been consolidated as one of the most effective strategies within the Bayesian approach. This method models the distributions of hyperparameters that lead to good and bad observations, allowing prioritization of regions of space with higher probability of improvement. Recent work has decomposed its internal components to better understand how each one influences the empirical performance of the algorithm [3].

The main objective of this research is to compare the performance of four derivative-free optimization methods when adjusting hyperparameters in machine learning models. The goal is to determine which method provides the best convergence in prediction errors while maintaining computational efficiency, that is, which one achieves the lowest prediction error without demanding too much time or resources.

II. Materials and Methods

A. Dataset

The study used the House Prices dataset from Kaggle, a regression problem widely recognized in the machine learning community. This dataset contains information about residential properties in Ames, Iowa, with 1,460 observations and 81 features that include both numerical and categorical variables.

The dataset characteristics encompass architectural aspects (total area, number of rooms, year built), land characteristics (lot area, topography), property conditions (overall quality, conservation status), and amenities (heating systems, garages, pools). The target variable corresponds to the sale price of the house, with a range from $34,900 to $755,000.

B. Descriptive Statistics

The database contains information on 1,460 houses with 80 features (37 numerical and 43 categorical), with the sale price (*SalePrice*) being the target variable. The average price was $180,921 with high dispersion (standard deviation of $79,443), reflecting significant heterogeneity in the real estate market.

TABLE I
SUMMARY OF RELEVANT DESCRIPTIVE STATISTICS

Variable	Mean	Range	Std. Dev.
SalePrice	$180,921	34,900 – 755,000	79,443
GrLivArea	1,515.5	334 – 5,642	–
LotArea	10,516.8	1,300 – 215,245	–
YearBuilt	1971.3	1872 – 2010	–
OverallQual	6.1	1 – 10	–

Variables like *GrLivArea* and *LotArea* present wide ranges, indicating the presence of houses with extreme characteristics. The *YearBuilt* variable shows a construction range between 1872 and 2010, while the overall quality (*OverallQual*) concentrates around a mean value of 6.1. These conditions support the choice of derivative-free optimization techniques, as it involves a noisy, expensive-to-evaluate objective function defined in a mixed search space.

The data used in this research was obtained from the Kaggle competition repository, specifically the dataset titled *House Prices - Advanced Regression Techniques*, which due to its attributes is a representative and challenging case for the application of advanced regression models and optimization techniques. The database is available at https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques/data.

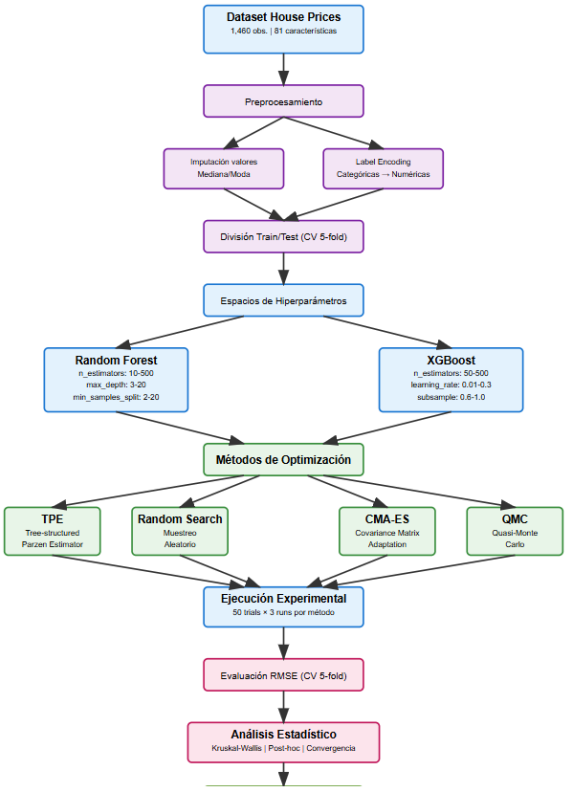

Fig. 1. Methodological Flow Diagram.

C. Data Preprocessing

Before subjecting the data to the model, we established a very clear cleaning and preparation protocol to ensure that each step was reproducible. First, we addressed missing values in numerical variables by substituting them with the median of the corresponding feature; this decision was made because the median is less sensitive to extreme outliers than the mean, which helps us preserve the statistical integrity of the dataset without distorting the original distribution. For qualitative variables, we applied similar logic but adapted to their nature, filling gaps with the mode of each attribute.

Finally, so that optimization algorithms could handle all data homogeneously, we converted categorical variables to numbers using Label Encoding. This approach allows optimization algorithms to work exclusively with numerical search spaces.

D. Machine Learning Models

Two representative algorithms were selected to evaluate the optimization methods: Random Forest and XGBoost. Random Forest is an ensemble method that combines multiple decision trees through average voting [6]. The optimized hyperparameters include the number of estimators (10-500), maximum depth (3-20), minimum samples for split (2-20), minimum samples per leaf (1-10), and feature selection criterion. The final prediction is calculated according to

$$\hat{y} = \frac{1}{B}\sum_{b=1}^{B} T_b(x) \tag{1}$$

where B is the total number of trees in the forest, and $T_b(x)$ represents the prediction of tree b for input x. XGBoost is a gradient boosting algorithm that builds models sequentially [7]. Its hyperparameter space is more complex, including number of estimators (50-500), learning rate (0.01-0.3), maximum depth (3-12), subsampling (0.6-1.0), feature subsampling (0.6-1.0), and L1 and L2 regularization parameters (0-2). In iteration t, the prediction for sample x_i is updated according to

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta f_t(x_i) \tag{2}$$

where η is the learning rate, and $f_t(x_i)$ is the prediction of the tree added in iteration t for sample x_i.

E. Derivative-Free Optimization Methods

Four representative derivative-free optimization methods were implemented: *Tree-structured Parzen Estimator (TPE), Random Search, Covariance Matrix Adaptation Evolution Strategy (CMA-ES), and Quasi-Monte Carlo (QMC).*

The Tree-structured Parzen Estimator (TPE) method adopts a sequential model-based optimization approach that builds approximations of hyperparameter performance based on historical measurements [3]. TPE models the distribution of hyperparameters using tree-structured Parzen estimators, separating observations into one for hyperparameter configurations

with good performance and another for those with poor performance, and chooses new candidates by maximizing the "Expected Improvement"

$$\text{EI}(\lambda) = \int_{-\infty}^{f^*} (f^* - f) \cdot p(\lambda \mid f)\, df \tag{3}$$

where f^* represents the best value observed so far, and $p(\lambda \mid f)$ is the conditional density of hyperparameters λ given performance f.

In contrast, Random Search does not build any intermediate model but simply samples uniformly from the hyperparameter space. Although simple, it provides a robust baseline and is particularly effective in high-dimensional spaces. For a continuous hyperparameter defined on interval $[a, b]$, the sample is generated as

$$\lambda \sim \text{Uniform}(a, b) \tag{4}$$

The Covariance Matrix Adaptation Evolution Strategy (CMA-ES) algorithm is an evolutionary method that dynamically adapts the covariance matrix of a multivariate normal distribution to guide the search toward promising regions of the solution space. Sample generation in iteration $g + 1$ is generated as

$$x_k^{(g+1)} \sim \mathcal{N}(m^{(g)}, (\sigma^{(g)})^2 C^{(g)}) \tag{5}$$

where $m^{(g)}$ is the mean, $\sigma^{(g)}$ is the step size, and $C^{(g)}$ is the covariance matrix corresponding to generation g.

Finally, Quasi-Monte Carlo (QMC) uses low-discrepancy sequences to achieve more uniform coverage of the search space compared to purely random methods. The quality of coverage is evaluated using the star discrepancy

$$D_N^* = \sup_B \left| \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}_B(x_i) - \text{Vol}(B) \right| \tag{6}$$

where $\mathbf{1}_B(x_i)$ is the indicator function of set B, and $\text{Vol}(B)$ is its volume.

The complete source code of the implementation and experiments performed is available in a public GitHub repository, facilitating its review and reproduction of results. It can be consulted at https://github.com/FYPC7/Paper_Derivative_Free_Optimization_for_Black_Box_Functions.git

F. Experimental Protocol

The experimental design followed a rigorous protocol to ensure statistical validity of results. Each optimization method was executed with 50 objective function evaluations, representing a balance between search space exploration and computational limitations.

For each method, 3 independent runs were performed with different random seeds, allowing analysis of variability and statistical significance. The objective function is defined as:

$$f(\lambda) = \text{RMSE}_{CV}(\lambda) = \sqrt{\frac{1}{K} \sum_{k=1}^{K} \frac{1}{|V_k|} \sum_{i \in V_k} (y_i - \hat{y}_i(\lambda))^2} \tag{7}$$

where $K = 5$ is the number of folds in cross-validation, V_k is the validation set of fold k, and $\hat{y}_i(\lambda)$ is the prediction with hyperparameters λ.

The evaluation of each hyperparameter configuration was performed using 5-fold cross-validation, using root mean square error (RMSE) as the performance metric. This methodology ensures robust model performance estimates and reduces the risk of overfitting.

III. Results

A. Performance of Optimization Methods in Random Forest

The performance analysis for Random Forest revealed very significant differences between the evaluated optimization methods. The results are presented in Table II, showing descriptive statistics for each method based on 3 independent runs.

TABLE II
COMPARATIVE PERFORMANCE OF DERIVATIVE-FREE OPTIMIZATION METHODS FOR RANDOM FOREST

Method	Avg RMSE	Std. Dev.	Best RMSE	Time (s)
TPE	29,803.67	316.86	29,355.86	66.7 ± 28.7
QMC	29,835.69	0.00	29,835.69	54.3 ± 7.6
Random Search	30,027.87	206.73	29,737.75	59.1 ± 5.9
CMA-ES	30,207.06	161.50	29,991.96	54.8 ± 12.3

TPE demonstrated the best average performance with an RMSE of 29,803.67, outperforming competing methods by statistically significant margins. Particularly notable is its ability to find the best individual solution (RMSE = 29,355.86), indicating effective exploration of the search space (Table II).

The QMC method showed interesting results with null variability between runs (standard deviation = 0.00), a characteristic expected due to its deterministic nature. However, its average performance was slightly inferior to TPE.

Random Search, used as a baseline, exhibited intermediate performance with moderate variability. CMA-ES, surprisingly, showed the worst average performance among the evaluated methods, possibly due to the specific characteristics of the Random Forest search space.

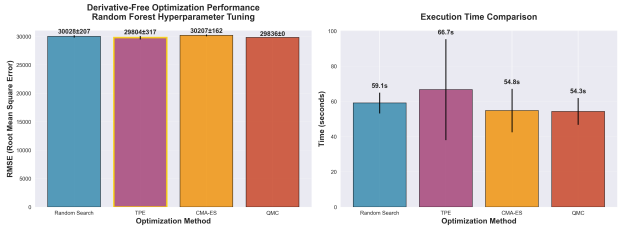

Fig. 2. Performance comparison between optimization methods.

B. Convergence Analysis

Analysis of convergence curves reveals distinct patterns in the search behavior of each method. TPE showed rapid convergence during the first 20 evaluations, followed by sustained gradual improvement. This behavior is consistent with its Bayesian nature, where initial evaluations inform the construction of the surrogate model.

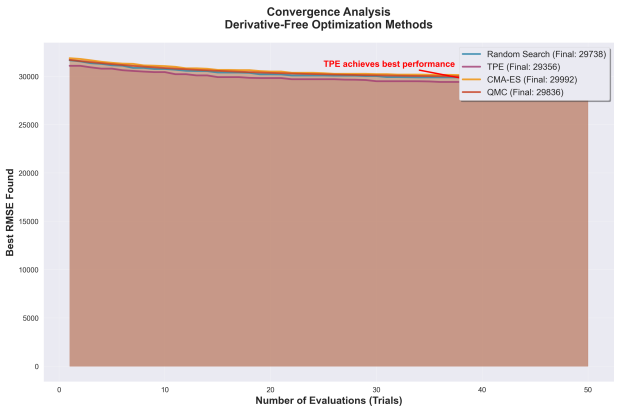

Fig. 3. Convergence curves of optimization methods

QMC exhibited smoother and more predictable convergence, resulting from its systematic sampling strategy. Random Search showed the typical expected variability, with occasional significant improvements followed by plateaus. CMA-ES presented slow initial convergence, possibly requiring more evaluations for effective adaptation of its covariance matrix.

C. Results in XGBoost

Experiments with XGBoost presented significant technical challenges. Most hyperparameter configurations resulted in execution errors or unstable numerical convergence, resulting in RMSE values of 50,000 (penalty value for failure). This behavior illustrates XGBoost's sensitivity to hyperparameter configuration and the importance of carefully designed search spaces.

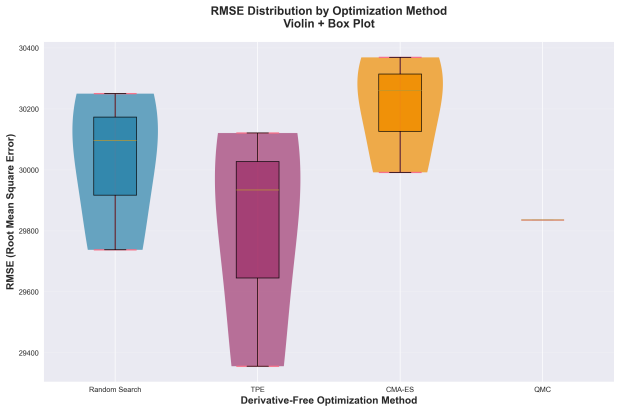

Fig. 4. Comparative analysis of performance between algorithms

Random Search was the only method that occasionally found stable configurations, although with performance significantly inferior to Random Forest results. These results suggest that XGBoost requires more sophisticated optimization strategies or more restrictive search spaces.

D. Comparative Analysis

Direct comparison between algorithms reveals the clear superiority of Random Forest optimized through TPE. The performance difference between Random Forest (RMSE = 29,803.67) and XGBoost (RMSE = 50,000) represents a 67.76% improvement, although this should be interpreted considering the technical difficulties encountered with XGBoost.

Computational efficiency analysis shows comparable execution times between methods for Random Forest, with TPE requiring slightly more time (66.7s) due to the construction and updating of surrogate models. However, this additional cost is justified by the improvement in quality of solutions found.

Fig. 5. Statistical Analysis

E. Statistical Analysis

Statistical significance tests confirm significant differences between methods. The Kruskal-Wallis test rejects the null hypothesis of equality of medians (p ¡ 0.05), validating that the observed differences are not the result of random variability.

$$H = \frac{12}{N(N+1)} \sum_{i=1}^{k} \frac{R_i^2}{n_i} - 3(N+1) \tag{8}$$

where H is the test statistic, N is the total number of observations, k is the number of groups, R_i is the sum of ranks of group i, and n_i is the size of group i.

Post-hoc analysis through pairwise comparisons reveals that TPE significantly outperforms Random Search and CMA-ES, while the difference with QMC, although favorable to TPE, does not reach statistical significance given the limited sample size.

IV. Discussion

The results obtained confirm the effectiveness of Bayesian optimization methods, specifically TPE, for hyperparameter tuning in machine learning algorithms. TPE allows starting with initial beliefs about the best model hyperparameters and updating these beliefs in a principled manner as one learns how different hyperparameters impact model performance [3].

TPE's superiority aligns with previous findings in the hyperparameter optimization literature. Empirical results consistently demonstrate its effectiveness across various application domains. QMC's deterministic behavior presents advantages in terms of reproducibility, but its slightly inferior performance suggests that TPE's probabilistic adaptation provides tangible benefits for hyperparameter space exploration.

The difficulties encountered with XGBoost illustrate an important challenge in hyperparameter optimization: the algorithm's sensitivity to specific configurations [7]. Not all combinations of hyperparameter values are compatible, creating hidden constraints. This suggests the need for more sophisticated strategies for constraint handling and search space definition.

CMA-ES's relatively poor performance in this context can be attributed to the mixed nature of the search space (continuous and categorical variables) and the limited size of the evaluation budget [4]. CMA-ES typically requires more evaluations for effective adaptation of its covariance matrix, especially in high-dimensional spaces.

The results also highlight the importance of rigorous experimental validation in evaluating optimization methods [10]. The variability observed between independent runs underscores the need for multiple repetitions for statistically valid conclusions.

The practical implications of these findings are significant for machine learning practitioners. TPE emerges as a robust and efficient option for automated hyperparameter tuning, providing a favorable balance between solution quality and computational efficiency [11].

V. Conclusions

This study provides empirical evidence of TPE's superiority over traditional derivative-free optimization methods for hyperparameter tuning in machine learning algorithms. The main findings include:

TPE achieved the best average performance (RMSE = 29,803.67) and found the best individual solution for Random Forest, demonstrating effectiveness in both exploration and exploitation of the search space. TPE's Bayesian nature enables efficient learning of the objective function structure, resulting in faster convergence compared to non-adaptive methods.

QMC showed advantages in terms of reproducibility and stability, although with slightly inferior performance to TPE. Its deterministic nature makes it valuable for applications where reproducibility is critical.

Random Search, although simple, maintains reasonable competitiveness and serves as an effective baseline. CMA-ES requires careful consideration of evaluation budget and search space characteristics for optimal performance.

The difficulties encountered with XGBoost emphasize the importance of careful search space design and handling of hidden constraints in hyperparameter optimization.

The implications of these results extend to machine learning practice, where the selection of optimization methods can significantly impact both final model quality and development process efficiency.

Future research should explore the adaptation of these methods for more complex search spaces, explicit constraint handling, and evaluation on more diverse datasets. Additionally, the development of efficiency metrics that balance solution quality with computational cost would represent a valuable contribution to the field.

References

[1] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2623–2631, 2019. DOI: https://doi.org/10.1145/3292500.3330701

[2] J. Larson, M. Menickelly, and S. M. Wild, "Derivative-free optimization methods," *Acta Numerica*, vol. 28, pp. 287–404, 2019. DOI: https://doi.org/10.1017/S0962492919000060

[3] S. Watanabe, "Tree-structured Parzen estimator: Understanding its algorithm components and their roles for better empirical performance," *arXiv preprint arXiv:2304.11127*, 2023. DOI: https://doi.org/10.48550/arXiv.2304.11127

[4] N. Hansen and A. Ostermeier, "Completely derandomized self-adaptation in evolution strategies," *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001. DOI: https://doi.org/10.1162/106365601750190398

[5] D. De Cock, "Ames, Iowa: Alternative to the Boston housing data as an end of semester regression project," *Journal of Statistics Education*, vol. 19, no. 3, pp. 1–14, 2011. DOI: https://doi.org/10.1080/10691898.2011.11889627

[6] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. DOI: https://doi.org/10.1023/A:1010933404324

[7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016. DOI: https://doi.org/10.1145/2939672.2939785

[8] H. Niederreiter, "Random number generation and quasi-Monte Carlo methods," *Society for Industrial and Applied Mathematics*, 1992. DOI: https://doi.org/10.1137/1.9781611970081

[9] F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Sequential model-based optimization for general algorithm configuration," *International Conference on Learning and Representation*, pp. 507–523, 2011. DOI: https://doi.org/10.1007/978-3-642-25566-34_0

[10] B. Bischl, M. Binder, M. Lang, T. Pielok, J. Richter, S. Coors, et al., "Hyperparameter optimization: Foundations, algorithms, best practices, and open challenges," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 13, no. 2, e1484, 2023. DOI: https://doi.org/10.1002/widm.1484

[11] L. Yang and A. Shami, "On hyperparameter optimization of machine learning algorithms: Theory and practice," *Neurocomputing*, vol. 415, pp. 295–316, 2020. DOI: https://doi.org/10.1016/j.neucom.2020.07.061

[12] M. Feurer and F. Hutter, "Hyperparameter optimization," in *AutoML: Methods, Systems, Challenges*, Springer, pp. 3–33, 2019. [Online]. DOI: https://doi.org/10.1007/978-3-030-05318-5_1

[13] Falkner, S., Klein, A., Hutter, F. (2018). BOHB: Robust and efficient hyperparameter optimization at scale. International Conference on Machine Learning, 2018. DOI: https://doi.org/10.48550/arXiv.1807.01774

[14] Bernd Bischl, Pascal Kerschke, Lars Kotthoff, Marius Lindauer, Yuri Malitsky, Alexandre Fréchette, Holger Hoos, Frank Hutter, Kevin Leyton-Brown, Kevin Tierney, Joaquin Vanschoren, "ASlib: A benchmark library for algorithm selection", 2016 DOI: https://doi.org/10.1016/j.artint.2016.04.003

[15] Snoek, J., Larochelle, H., Adams, R. P.. "Practical Bayesian optimization of machine learningalgorithms". Advances in Neural Information Processing Systems, 2012 DOI: https://doi.org/10.48550/arXiv.1807.01774